

Cognitive Walkthrough

Evaluationskontext

<u>Software</u>	LLM Interaction Management System (LIMS)	
<u>Version</u>	1.0.0	
<u>Betriebssystem</u>	Windows 10 22H2	
<u>Nutzer</u>	Proband 4	
<u>Nutzerprofil</u>	<u>Erfahrung mit Programmierung</u>	<u>Erfahrung mit Python</u>
(Selbsteinschätzung)	6/10	3/10
<u>Testdatum</u>	05.08.2026	

T1	Endnutzerdokumentation aus dem GitHub-Repository beziehen
Ziel	Endnutzerdokumentation auf dem Zielsystem bereitgestellt, für weitere Hilfen geöffnet und als Anleitung für weitere Tasks bereit
Vorbedingung	Das GitHub-Repository ist bekannt
Schritt 1:	
Aktion	Der Nutzer bezieht die Endnutzerdokumentation aus dem Repository
Antwort	-
Nutzerverhalten	Der Nutzer orientiert sich zunächst an der Struktur des GitHub-Repositories und prüft die vorhandenen Dateien. Anschließend nutzt er den Verweis im README, um die Endnutzerdokumentation zu öffnen.

T2	Software aus dem GitHub-Repository herunterladen und in der gewünschten Python-Umgebung installieren
Ziel	Die Software ist in der gewünschten Umgebung vorhanden und könnte in einem Projekt importiert werden
Vorbedingung	Die Nutzerdokumentation ist zur Unterstützung geöffnet und bereit, der Nutzer kann in der Dokumentation den Ablauf der Installation abrufen und nachvollziehen
Schritt 1: Befehl zur Installation ausführen	
Aktion	pip-Kommando aus der Endnutzerdokumentation herauslesen und in einer Eingabeaufforderung der Python-Umgebung ausführen
Antwort	-
Nutzerverhalten	Der Nutzer öffnet gezielt die Eingabeaufforderung des Betriebssystems und überprüft zunächst mit python --version, ob Python verfügbar ist. Anschließend übernimmt er den in der Endnutzerdokumentation angegebenen pip-Befehl und führt die Installation der Schnittstelle aus.
Schritt 2: Installation überprüfen	
Aktion	Installation beispielsweise durch eine Versionskontrolle in einem kleinen Codebeispiel überprüfen
Antwort	Python-Umgebung gibt Version des Tools aus
Nutzerverhalten	Der Nutzer übernimmt das in dem Repository gezeigte Beispiel zur Versionsabfrage in eine Python-Umgebung und überprüft damit erfolgreich die Installation der Schnittstelle.

T3	Initialisierung der Software durch Objekt oder API
Ziel	Die Software ist in ihrem „Idle-Zustand“, mit den externen Handlern verbunden und der Nutzer hat entweder ein InteractionManager-Objekt oder eine Referenz zur High-Level API
Vorbedingung	T2 erfüllt
Schritt 1: Verbindungsstrukturen erstellen	
Aktion	Bei einer ersten Kommunikation mit den Endpunkten müssen die Datenstrukturen die Kommunikationsinformationen anbieten. Diese müssen vom Nutzer übergeben werden
Antwort	Keine Antwort des Systems
Nutzerverhalten	Der Nutzer öffnet ein neues Projekt in Visual Studio Code und übernimmt die Beispielinitalisierung aus der Endnutzerdokumentation. Dabei verwendet er die Autovervollständigung der Python-Erweiterung, um die benötigten Importe und Methoden zu identifizieren, und passt die Verbindungsdaten an seine Konfiguration an.
Schritt 2: Verbindung initialisieren	
Aktion	Der Nutzer übergibt die Verbindungsinformationen an die Schnittstelle und startet den Verbindungsaufbau.
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer orientiert sich weiterhin an der Endnutzerdokumentation, erstellt ein InteractionManager-Objekt und führt die Methode zum Verbindungsaufbau auf
Schritt 3: Verbindung überprüfen	
Aktion	Nutzer verwendet eine der Methoden, beispielsweise <code>is_connected()</code> , um die Verbindung zu überprüfen
Antwort	System gibt Rückmeldung über Verbindungsstatus
Nutzerverhalten	Der Nutzer sieht <code>is_connected()</code> in der Autovervollständigung und verwendet die Methode ohne weitere Dokumentation zu betrachten.

T4	Erneute Initialisierung basierend auf vorherigen Einstellungen
Ziel	Der Nutzer initialisiert die Software bei einer erneuten Verwendung aus den gespeicherten Verbindungsdaten, also ohne explizite Auswahl von Schnittstellen oder Angabe von Verbindungsinformationen
Vorbedingung	T2 erfüllt
Schritt 1: Verbindung initialisieren	
Aktion	Nutzer startet ohne Informationen einen Verbindungsaufbau
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer verwendet den zuvor erstellten Code erneut und entfernt die explizite Übergabe der Verbindungsinformationen. Anschließend führt er den Code erneut aus.
Schritt 2: Verbindung überprüfen	
Aktion	Nutzer verwendet eine der Methoden, beispielsweise <code>is_connected()</code> , um die Verbindung zu überprüfen
Antwort	System gibt Rückmeldung über Verbindungsstatus
Nutzerverhalten	Da der Code noch <code>is_connected()</code> enthält, erhält der Nutzer gleich eine Rückmeldung über die erfolgreiche Verbindung.

T5	Prompt senden und Antwort erhalten
Ziel	Der Nutzer versteht die Interaktion zum Senden eines oder mehrerer Prompts zum LLM und kann die Antworten erhalten. Persistente Daten dieses Tasks sind ebenfalls für weitere Tasks erforderlich
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Konversation starten	
Aktion	Nutzer verwendet die Methode <code>start_conversation()</code> zum Starten einer Konversation
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer erweitert den in den vorherigen Tasks erstellten Code um den in der Endnutzerdokumentation beschriebenen Aufruf von <code>start_conversation()</code> .
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer ergänzt den bestehenden Code um den Aufruf von <code>send_prompt()</code> entsprechend der Endnutzerdokumentation und führt ihn aus. Da zunächst keine Ausgabe erfolgt, erkennt er selbstständig, dass der Rückgabewert mit <code>print()</code> ausgegeben werden muss. Anschließend erhält er bei erneuter Ausführung die Antwort des LLM.

T6	Setzen eines Systemprompts, erneutes Senden eines Prompts
Ziel	Der Nutzer weiß, wie Einstellungen vorgenommen und ausgewählt werden, ein Systemprompt ist hinterlegt und dessen Einfluss kann bei einer weiteren Nachricht überprüft werden
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Hinterlegen eines Systemprompts	
Aktion	Hinzufügen von Daten zur Einstellung „ <code>system_prompt</code> “
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer durchsucht die Endnutzerdokumentation gezielt nach dem Begriff <i>Systemprompt</i> und findet die Einstellung <code>system_prompt</code> . Anschließend erweitert er seinen bestehenden Code um den Aufruf von <code>write_setting()</code> , um den Systemprompt dauerhaft zu hinterlegen.
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt mit hinterlegtem Systemprompt zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer verwendet den bereits in den vorherigen Aufgaben erstellten Code zum Senden eines Prompts erneut und überprüft anhand der Antwort des LLM, dass der zuvor hinterlegte Systemprompt berücksichtigt wurde.

T7	Hinzufügen von Kontextdaten, erneutes Senden eines Prompts
Ziel	Der Nutzer weiß, wie komplexe Kontextdaten an den Prompt angefügt werden können und kann die Kontext-Einstellungen unterscheiden
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“
Schritt 1: Kontextdaten hinzufügen	
Aktion	Hinzufügen von Kontextdaten durch die Methode <code>set_context_data()</code>
Antwort	Keine direkte Antwort des Systems
Nutzerverhalten	Der Nutzer findet die Methode <code>set_context_data()</code> in der Endnutzerdokumentation und erweitert seinen bestehenden Code entsprechend. Für den Aufbau des benötigten dict orientiert er sich an den zuvor verwendeten Datenstrukturen.
Schritt 2: Prompt senden	
Aktion	Nutzer verwendet die Methode <code>send_prompt()</code> , um ein Prompt mit hinterlegten Kontextdaten zu senden
Antwort	System sendet Antwort des LLM
Nutzerverhalten	Der Nutzer verwendet den bereits vorhandenen Code zum Senden eines Prompts erneut und überprüft anhand der Antwort des LLM, dass die hinterlegten Kontextdaten in die Kommunikation einbezogen wurden. Er erkennt ebenfalls sichtbar, dass der Systemprompt ebenfalls noch verwendet wird.

T8	Ähnlichkeitssuche auf der Vektordatenbank
Ziel	Der Nutzer weiß, wie eine Ähnlichkeitssuche auf Basis der vorherigen Ergebnisse durchgeführt wird. Die Antwort der Schnittstelle erscheint im Kontext der vorherigen Tasks subjektiv logisch
Vorbedingung	Software initialisiert und verbunden – „Idle-Zustand“, Daten vorhanden
Schritt 1: Ähnlichkeitssuche ausführen	
Aktion	Nutzer verwendet die Methode <code>nearest_search_vector</code> um eine Ähnlichkeitssuche auszuführen
Antwort	<code>top_k</code> ähnliche Einträge zur Eingabe werden aus der Vektordatenbank ausgegeben
Nutzerverhalten	Der Nutzer findet die Methode <code>nearest_search_vector()</code> in der Endnutzerdokumentation und ergänzt seinen bestehenden Code um den Methodenaufruf. Die Bedeutung des Parameters <code>top_k</code> erschließt er sich aus der Methodendokumentation im Code. Anschließend ergänzt er eigenständig eine <code>print()</code> -Ausgabe, führt den Code aus und erhält eine kleinere Anzahl von Treffern. Da im Rahmen des Cognitive Walkthroughs bislang nur wenige Konversationen gespeichert wurden, erkennt er, dass sämtliche verfügbaren Ergebnisse zurückgegeben wurden.